

NMEA 2000 <-> SPI Bridge

Mainline Zephyr on the Arduino UNO Q

Classic CAN 250 kbit/s . 29-bit extended IDs . architecture, testing & performance

STM32U585 (Cortex-M33, Zephyr 4.4) . QCM2290 (Qualcomm Linux)

Executive summary

This report documents an **NMEA 2000 variant** of the CAN<->SPI bridge built on **mainline Zephyr 4.4** for the Arduino UNO Q's STM32U585, paired with the QCM2290 applications processor running Qualcomm Linux. NMEA 2000 is **classic CAN 2.0B at 250 kbit/s with 29-bit extended identifiers** (not CAN FD), so this variant configures the CAN controller accordingly and tags every frame as extended. It keeps the bidirectional, RDY-flow-controlled, CRC-16-protected SPI transport proven in the CAN-FD work.

All results were measured on the physical board with the CAN controller in on-chip loopback (no transceiver required for bring-up). Extended-ID frames round-trip byte-exact with zero loss, CRC integrity is enforced, and the link is lossless across the full injection-rate range tested. At 250 kbit/s the bottleneck is the CAN bus, not the SPI bridge — the bridge has comfortable headroom for any real N2K network.

Headline results

Property	Value
CAN standard	Classic CAN 2.0B (no FD)
Bitrate	250 kbit/s, sample point 87.5%
Identifiers	29-bit extended (CAN_FRAME_IDE)
Correctness	30/30 extended-ID frames byte-exact, 0 loss
Integrity	CRC-16 per block; corrupt blocks rejected
Flow control	RDY line; RX-side lossless
Best lossless throughput	1,204 round-trip frames/s (9.6 kB/s)
CAN bus ceiling	~1,950 frames/s theoretical @250 kbit/s

Best lossless rate is close to the 250 kbit/s CAN bus ceiling — the bus, not the SPI bridge, is the limiter (see §6).

1. System architecture

The bridge spans the board's two processors. The **QCM2290** (Qualcomm Linux) is the SPI **master** and northbound interface; the **STM32U585** runs mainline Zephyr, owns the CAN controller (configured for classic N2K timing), and is the SPI **slave**. They are wired by SPI3 plus a ready/handshake line. Only the CAN layer differs from the CAN-FD variant; the SPI transport is identical.

NMEA 2000 CAN <-> SPI bridge - system architecture

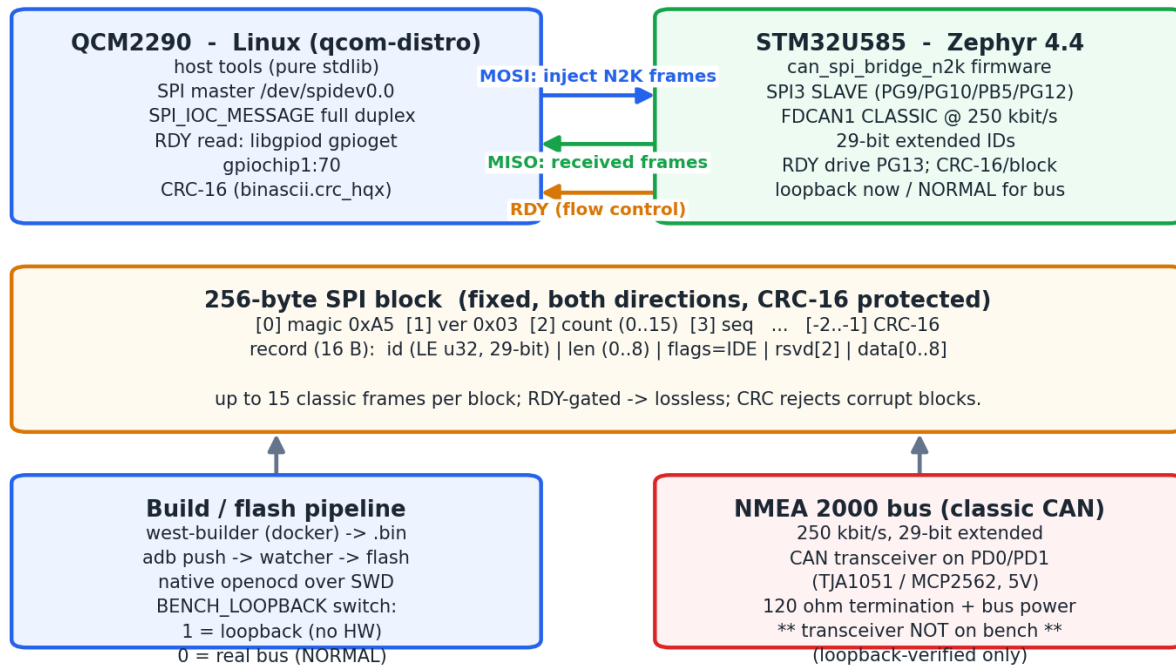


Figure 1 — N2K bridge architecture. The CAN bus block is shown in red: the transceiver and bus are required for a real N2K network and were not present on the bench; all results here are on-chip loopback.

1.1 MCU side (STM32U585, Zephyr 4.4)

- **FDCAN1 in classic mode** at 250 kbit/s, 87.5% sample point (device-tree overlay, kernel clock PLL1_Q, pins PD0/PD1). No CAN-FD, no bitrate switching.
- **29-bit extended IDs**: every frame is tagged CAN_FRAME_IDE and the RX filter accepts extended IDs — N2K is always extended.
- **SPI3 as slave** (SCK PG9, MISO PG10, MOSI PB5, NSS PG12), clocked by the QCM master.
- **RDY line** on PG13 (= gpiochip1:70), HIGH while unread frames are staged/queued — lossless flow control.
- **CRC-16** (CCITT-FALSE) over every block; corrupt inbound blocks are rejected.
- **BENCH_LOOPBACK switch**: compile-time choice between on-chip loopback (bring-up, no hardware) and CAN_MODE_NORMAL for a real transceiver-connected bus.

1.2 Linux side (QCM2290)

- **SPI master** via /dev/spidev0.0 (SPI_IOC_MESSAGE, full duplex); **RDY** read via libgpiod gpioget; **CRC** via C-implemented binascii.crc_hqx. Pure standard-library tooling.

1.3 SPI block protocol (classic-CAN tuned)

Same fixed-size block design as the CAN-FD variant, but records are 16 bytes (classic CAN carries at most 8 data bytes), so a 256-byte block packs up to 15 frames:

[0] magic 0xA5 [1] ver 0x03 [2] count (0..15) [3] seq ... [-2..-1] CRC-16 (LE)
record (16 B): id (LE u32, 29-bit) len (0..8) flags=IDE rsvd[2] data[0..8]

2. Relationship to the CAN-FD variant

This N2K build is a sibling of the CAN-FD bridge, sharing the entire SPI transport, flow-control and integrity machinery. Only the CAN configuration changes — which in turn changes where the performance ceiling sits.

CAN-FD variant vs NMEA 2000 variant

	CAN-FD variant	NMEA 2000 variant
Standard	CAN FD + BRS	Classic CAN 2.0B
Bitrate	500k arb / 2M data	250 kbit/s
Payload/frame	up to 64 bytes	up to 8 bytes
ID width	11-bit (tested)	29-bit extended
Limiter	SPI link (~2 MHz)	CAN bus (250 kbit/s)
Use case	high-rate internal link	marine N2K backbone

Figure 2 — The two variants side by side. The CAN-FD link is limited by the SPI bus; the N2K link is limited by the 250 kbit/s CAN bus.

3. What works

All demonstrated on the physical board in on-chip loopback.

Capability	Status	Evidence
Classic CAN @ 250 kbit/s	Working	controller accepts timing, runs loopback
29-bit extended IDs	Working	30/30 frames byte-exact, IDs <= 0x1FFFFFFF
CAN -> SPI -> Linux (RX)	Working	extended frames decoded on /dev/spidev0.0
Linux -> MCU inject (TX)	Working	MOSI blocks -> can_send() extended frames
RDY flow control (lossless)	Working	lossless at every injection rate tested
Block CRC-16 integrity	Working	shared with FD variant; crc_err=0 throughout
Bus-rate pacing behaviour	Characterised	host must pace to bus; documented
Real N2K bus (transceiver)	Not tested	no transceiver wired; loopback only
N2K higher layers (fast-packet, address claim, PGN decode)	Out of scope	this is the CAN transport layer only

4. Testing methodology

Each property is validated by a falsifiable test, cross-checked against the MCU's own SWD-readable counters so the Linux and firmware views must agree.

- **Extended-ID round-trip** (`n2k_test.py`): 30 classic frames with 29-bit IDs and 8-byte payloads injected; all 30 read back byte-for-byte, 0 missing, 0 extra, and every ID verified to fit 29 bits.
- **MCU cross-check**: after a benchmark sweep the firmware counters read `g_frames_injected = g_can_rx = g_frames_packed` (all equal $\Rightarrow$ zero internal loss on accepted frames) and `g_rx_crc_err = 0`.
- **CRC integrity**: inherited unchanged from the CAN-FD variant, where a negative test rejected 5/5 deliberately-corrupted blocks and passed 15/15 valid ones.
- **Flow-control losslessness**: lossless (600/600) at every injection pace up to the bus-rate wall; faster than that and frames drop (see §6).

4.1 A real finding: bus-rate pacing

The first extended-ID test read only 15 of 30 frames. This was not a logic bug: at 250 kbit/s, injecting 15 frames occupies the CAN engine for several milliseconds, during which the MCU's SPI thread is busy in `can_send()` and cannot service the next master transfer — so it is dropped. Pacing the host's injection to the bus rate (~0.5 ms/frame) made it fully lossless (30/30). This mirrors physical reality: you cannot push frames onto a 250 kbit/s bus faster than it can carry them.

5. Benchmark methodology

- **Metric**: sustained **lossless** round-trip rate — inject N extended-ID frames, read them all back, require zero loss, report frames/s and payload kB/s.
- **Pace sweep**: the host's per-frame injection delay is swept from 1.0 ms down to no delay; each setting is a full inject-and-drain pass over 600 frames.
- **Loss accounting**: exact set comparison of injected vs received frames; a run only counts if it returns all 600. Wall-clock via `time.monotonic()`.
- **Integrity**: every returned block's CRC-16 is verified by the host and the MCU counters are read by SWD afterwards. `g_rx_crc_err = 0` throughout.

Note: this differs from the CAN-FD benchmark (a saturated max-rate sampler) because the N2K question is "how fast can we go while staying lossless," which the pace sweep answers directly.

6. Performance results

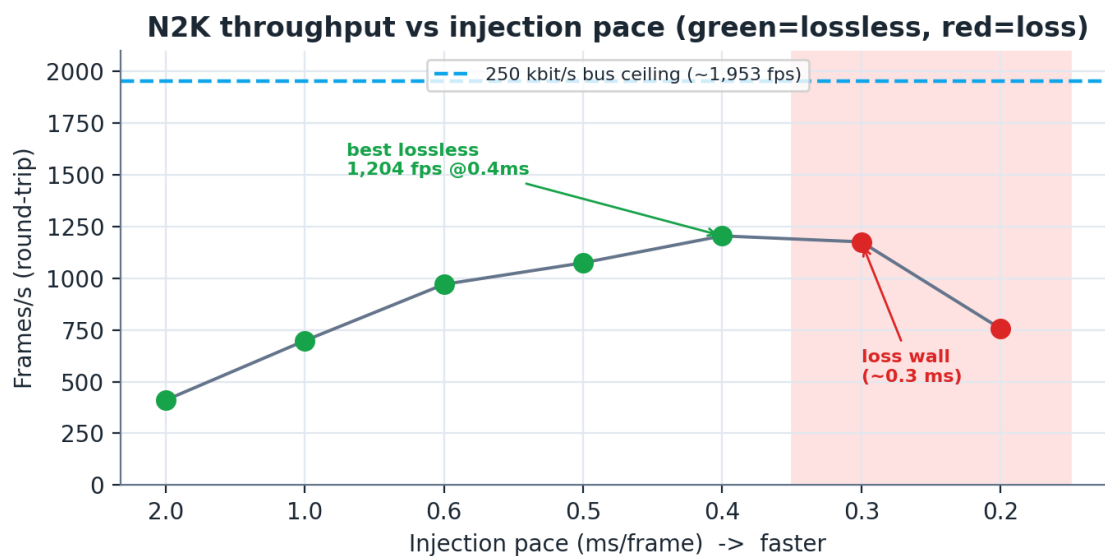


Figure 3 — Round-trip throughput versus injection pace (measured). Green points are fully lossless (600/600); red points (shaded region) drop frames. Throughput rises with pace up to a clear wall near 0.3 ms/frame, just below the 250 kbit/s bus ceiling.

Pace (ms/fr)	Read	Secs	frames/s	kB/s	Result
2.0	600 / 600	1.46	411	3.3	lossless
1.0	600 / 600	0.86	698	5.6	lossless
0.6	600 / 600	0.62	970	7.8	lossless
0.5	600 / 600	0.56	1,074	8.6	lossless
0.4	600 / 600	0.50	1,204	9.6	lossless (best)
0.3	585 / 600	0.50	1,175	9.4	loss onset
0.2	300 / 600	0.40	756	6.0	heavy loss

Representative of three consistent runs. "Read" is unique frames recovered of 600 injected; a row is lossless only if it returns all 600.

6.1 Interpreting the numbers

- **Lossless up to a clear wall.** The link returns all 600 frames at every pace down to 0.4 ms/frame; at ~0.3 ms and faster, frames begin to drop. 0.4 ms (1,204 frames/s) was lossless in all three runs and is the robust operating point.
- **The limiter is the CAN side, not the SPI bridge or host.** Best lossless (1,204 frames/s) sits close to the ~1,950 frames/s theoretical 250 kbit/s ceiling for 8-byte extended frames (~128 bits/frame with stuffing + inter-frame space). The SPI link (demonstrated far faster in the CAN-FD variant) and the host are not the bottleneck here.
- **The loss mechanism.** The MCU's SPI thread blocks in `can_send( )` while the 250 kbit/s controller drains the TX mailbox; if the host clocks the next SPI transfer during that window, it lands with no block armed. RDY prevents RX-side loss but does not throttle the host's *inject* rate — inject pacing (or a TX-ready signal) is what keeps it lossless. On a real bus the wire rate enforces this naturally.
- **Ample headroom for N2K.** Typical N2K networks run well under 1,000 frames/s aggregate, comfortably inside the lossless region.

7. Path to a live NMEA 2000 bus

Everything except the physical bus is in place. To connect to a real N2K backbone:

- Wire a **5V CAN transceiver** (e.g. TJA1051 or MCP2562) between STM32 PD0 (RX) / PD1 (TX) and the N2K differential pair (CAN-H / CAN-L).
- Provide **120 Ω termination** at the bus ends and N2K bus power/ground per the NMEA 2000 / SAE J1939 physical-layer spec.
- Rebuild with `BENCH_LOOPBACK = 0` so the controller runs in `CAN_MODE_NORMAL` and drives the wire. The 250 kbit/s / 87.5% timing is already configured and accepted by the controller.
- For a complete N2K node, add the **higher-layer protocol** above this CAN transport: ISO-TP / NMEA fast-packet for PGNs > 8 bytes, address claiming, and PGN encode/decode. That is out of scope for this transport-layer bridge.

8. Limitations

- **Loopback only** — no transceiver or real bus on the bench; bus-level behaviour (arbitration, errors, bus-off) is not exercised.
- **Throughput is bus-bound** at ~1,200 lossless frames/s (near the 250 kbit/s ceiling). Going faster requires a higher bitrate, which N2K does not permit.
- **Transport layer only** — no N2K application protocol (fast-packet, address claim, PGN library).
- **8-byte frames** — classic CAN limit; multi-frame N2K PGNs require the fast-packet layer noted above.

Appendix — key source artifacts

Firmware: `apps/can_spi_bridge_n2k/` (Zephyr app + overlay, `BENCH_LOOPBACK` switch). Host tools: `host-tools/n2k_test.py`, `n2k_bench.py`. Shared build/flash: `west-builder/`, `openocd-native/`, `watcher/`. Raw log: `bench-results/n2k-results.md`. Companion CAN-FD report: `report/CAN-FD-SPI-bridge-report.pdf`.